University of Kansas

Arithmetic Expression Evaluator C++
Software Architecture Document

Version 1.0

Revision History

Date	Version	Description	Author*
02/Apr/26	0.1	Section 2	Austin Haight
04/Apr/26	0.2	Section 1.1 and 1.2	Eric Faltermeier
05/Apr/26	0.3	Added content for sections 1.5, 5.1, 5.2.	John P.
06/Apr/26	0.4	Added Section 8 (Quality) and updated document formatting.	Yecolia E.
07/Apr/26	0.5	Added content for section 6	Tugs T.
09/Apr/26	0.6	Added sections 3 and 5	Alexaxnder W.
12/Apr/26	1.0	Finalized formatting and remaining sections.	John P.

* See team details below

Team Details:

- John Pannell – Programmer – j684p096@ku.edu
- Austin Haight – Repository Handler, Programmer – austindhaight@ku.edu
- Tugs-Oyut Tsedensodnom – Programmer – t226t008@ku.edu
- Eric Faltermeier – Meeting Lead, Programmer – e559f616@ku.edu
- Alexander Wichern – Notetaker, Programmer – alexanderwichern@ku.edu
- Yecolia Ephraim – Programmer, Notetaker – yecolia.ephraim@ku.edu

© University of Kansas, 2026

Table of Contents

Software Architecture Document

1. Introduction

1.1 Purpose

This Software Architecture Document provides an overview of the system's structure, design, and organization. It describes how the system is divided into components, how those components interact, and the key decisions made during development. The document is intended to help developers, instructors, and stakeholders understand how the system works at a high level.

It includes architectural views, design considerations, and explanations of the system's overall functionality. This document will be used as a reference throughout development and future updates.

1.2 Scope

This document applies to the entire system and describes its overall architecture and design. It focuses on the main components of the system, their interactions, and the structure used to support the system's functionality.

It influences how the system is developed, organized, and maintained. The document does not go into detailed implementation or code-level specifics but instead provides a high-level view of how the system is designed.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SAD	Software Architecture Document that describes the overall structure, components, and design of the system.
Tokenizer	Component that converts the input expression into tokens.
I/O	Input/Output that shows the input data received and the output data produced in a system.

Refer to the Project-Glossary.md.

1.4 References

- **Arithmetic Expression Evaluator Project Glossary**, Version 1.2, April 2026:

  Project glossary defining key words, abbreviations, and acronyms. Available in *Project-Glossary.md.*

1.5 Overview

The SAD describes the structure of the architecture and key components for the arithmetic expression evaluator system developed in C++. The document is organized to explore the architectural design of the system and explains how the software requirements will impact architectural decisions.

- **Section 1 – Introduction:** Establishes the purpose, scope, key terms, references, and context for the SAD and its subsections.
- **Section 2 – Architectural Representation:** Describes the architectural representation for the system and the different views to represent the system structure.
- **Section 3 – Architectural Goals and Constraints:** Outlines the effect that software requirements and constraints, such as support of binary and unary operators, will have on the architectural goals for the system.
- **Section 4 – Logical View:** Provides a detailed breakdown of the system into major packages, such as the parser, evaluator, and error handler, along with their responsibilities and interactions.

© University of Kansas, 2026

- **Section 5 – Interface Description:** Describes how the users interact with the system through a command-line interface, including the process to inputs and expected outputs.
- **Section 6 – Quality:** Discusses quality attributes, such as reliability and portability, and how the system architecture supports these characteristics.

2. Architectural Representation

The system follows a modular architecture design approach with layers defining the high-level structure. The system is decomposed into a set of components that has a specific responsibility for a stage in the process of evaluation. These modules are designed to be loosely coupled, while still communicating through well-defined interfaces. The primary focus is the Logical View, which highlights the decomposition of the system using UML diagrams to clearly illustrate the components and their relationships.

This section further provides a high-level decomposition into the major components and briefly outlines their responsibilities. Detailed implementation logic is further expanded in the Logical View.

2.1 User I/O

- A command line user interface.
- Handles getting user inputs and displaying requested outputs to the screen.
- Responsible for all string conversions that pertain to output information (syntax tree drawing, error notifications, etc.)
- Retrieves user inputs as strings and performs no preprocessing on expressions.

2.2 Tokenizer

- Takes in expression strings and parses them into token format.
- Returns a pointer to an array of Token objects for each expression string. Token arrays are in preorder notation.
- Recurses on parentheses.
- On encountering a syntax error, generates an error token at the end of the array and immediately returns.

2.2.1 Token Object Types

The Token class is a virtual class used to hold the values and operations of an expression. Tokens contain an integer that represents the number of inputs the token requires, and a basic toString method that returns the typeOf result of the class. Subclasses of Token are as follows:

- Integer – 0 inputs, contains an integer, overrides toString to return the stored value
- Float – 0 inputs, contains a float, overrides toString to return the stored value
- Parentheses – 0 inputs, contains an array of Tokens, overrides toString to return the toStrings of the array contents appended together and surrounded by parentheses
- Operator – 2 inputs, contains an integer representation of the operation being performed, overrides toString to return the string representation of the operation represented
- Negation – 1 input

2.3 Parser

- Takes in a Token array pointer.
- Checks the ordering of Tokens to make sure each has the correct number of inputs to consume.
- Re-orders the Tokens in the given array to follow PEMDAS evaluation order.
- Recurses on parentheses Tokens.
- Returns error state as Boolean.

2.4 Evaluator

- Takes in a Token array pointer.
- Evaluates the Tokens recursively.

 © University of Kansas, 2026

- Returns an array of 3 floats representing in order: calculated value, error type, error location index
- Immediately returns when encountering any error, does not attempt to find all errors in one run.

2.5 Program Controller

- Stores and moves data between above modules.
- Responsible for calling module methods and handling their outputs.

3. Architectural Goals and Constraints

The software will function as an arithmetic expression parser, evaluating inputs in the correct order (PEMDAS) and with proper parentheses grouping, as well being able to detect a variety of syntax and mathematical errors. The system must also detect potential errors in the input, such as division by zero, while producing clear and informative output for the user.

The program will be implemented entirely in C++ using an object-oriented design philosophy and taking input/displaying outputs via command line interface. The program's functionality will be based on its ability to pass a series of team-developed unit tests.

The goals and constraints for the arithmetic expression evaluator impact the style and design for the system’s architecture. Requirements for performance, usability, and reliability force a modular system structure. As a result, the separation between input handling, tokenization, parsing, evaluation, and error handling must be designed in the system's architecture to define clear responsibilities that support these goals.

4. Logical View

4.1 Overview

The arithmetic expression evaluator system follows a modular design in which it is decomposed into a set of cohesive modules or packages. Each module has a specific responsibility for a defined stage in the process of the system. These modules must also interact with one another to transform the user input into a computed result.

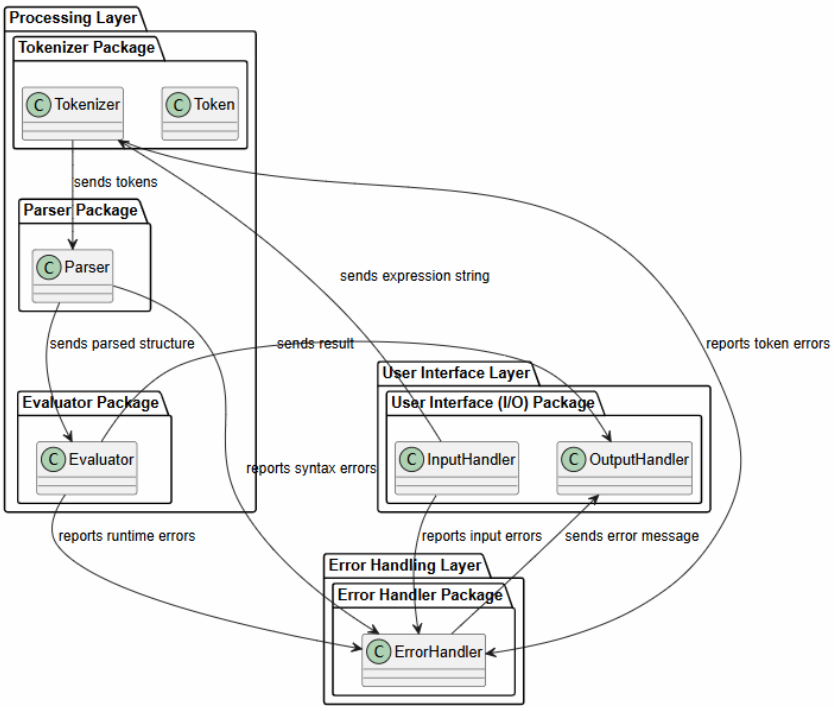
The system is organized into the following layers:

- **User Interface Layer:** Handles the user interaction through a command-line interface that accepts input from the user and displays the corresponding result.
    - **User Interface (I/O) Package:** Accepts arithmetic expressions from the user and gives the inputs to the Processing layer. Displays the computed results or error message to the user based on the result from the Processing or Error Handling layers.
- **Processing Layer:** Holds the core logic of the system, including tokenization, parsing, and evaluation of the expressions.
    - **Tokenizer Package:** Translates the input string into a sequence of tokens for the numbers, operators, and parentheses that may have been entered.
    - **Parser Package:** Analyzes tokens to ensure the order of operations (PEMDAS) is followed and generates a structured representation of the expression to be evaluated.
    - **Evaluator Package:** Computes the result of the parsed expression based off order of operations, associativity, and parentheses.
- **Error Handling Layer:** Detects and reports errors in any of the other layers, such as detecting invalid expressions.
    - **Error Handler Package:** Monitors all layers to detect any error for the system and communicates with Output Handler to display the error.

The flow of interaction is as follows:

 © University of Kansas, 2026

1. The user inputs an arithmetic expression using the command-line interface in the **User Interface** layer.
2. The **Processing** layer begins with the **Tokenizer** converting inputs into tokens.
3. The **Parser** creates a structured representation based off tokens.
4. The **Evaluator** creates the result.
5. The **User Interface** layer will display the results or errors.
6. Any errors that are found during tokenization, parsing, or evaluation are handled by the **Error Handling** layer.



SYSTEM DIAGRAM:

Each box in the UML diagram represents a system component, such as a layer, package, or class. Each arrow in the diagram represents the flow of data between components, and the process of interaction between components.

4.2 Architecturally Significant Design Modules or Packages

4.2.1 User Interface (I/O) Package

Handles reading arithmetic expressions from the user and advancing them to the packages in the Processing layer. This package is the connection point between the user and the processing logic for the expression that is entered. Basic input checks, such as empty input or illegal characters, are also performed.

Significant Classes:

- **InputHandler**: Reads user input from the command line, makes basic input checks, and passes it to the Tokenizer for further processing. This class is responsible for accepting arithmetic expressions entered by the user and is considered the point of interaction between the user and the system.
    - **Key Attributes/Operations:** There will be a string expression attribute to store user input. A read expression operation will read the expression from the user, and a basic validate input operation will be used for small error checking.
- **OutputHandler**: Displays the computed result or error message to the user through the command-

line interface. Interacts with the Processing layer (Evaluator Package) to receive the result of the computation and with the Error Handler if there is an error in the input.

- **Key Attributes/Operations:** Result and error message string attributes are used to store the result of the expression or the error message to display. There will also be operations to display the result and error messages.

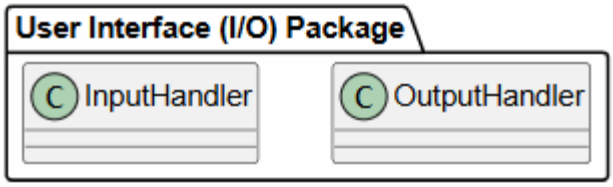
The 

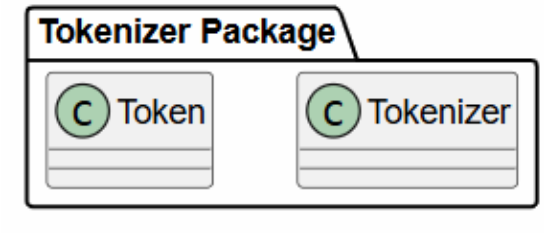
diagram shows the User Interface (I/O) package containing the classes that handle user input and output.

4.2.2 Tokenizer Package

Converts the input arithmetic expression into a sequence of tokens that may contain numbers, operators, and parentheses. This package is connected from the Input Handler to receive the input that the user entered. After the sequence of tokens has been created, the results are sent to the Parser. The main responsibility is to break down the input, so it can be properly interpreted in later packages.

Significant Classes:

- **Tokenizer**: Generates tokens for numbers, operators, and parentheses to allow the Parser to retrieve tokens for analysis. It ensures that all inputs are tokenized properly and ready for further processing.
  - **Key Attributes/Operations**: A tokens array attribute will hold the list of tokens. A key operation is the tokenize method that converts the string to tokens.
- **Token**: Represents one single token or piece in the expression. Each token relies on a type and value attribute.



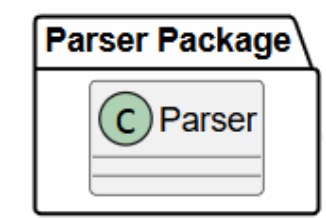
This diagram shows the Tokenizer package containing the classes responsible for converting the input into a sequence of tokens.

4.2.3 Parser Package

Creates a structured representation of the arithmetic expression to allow for operator precedence, associativity, and parentheses to be properly interpreted. The Parser gets the input tokens from the Tokenizer and will eventually be sent to the Evaluator.

Significant Classes:

- **Parser**: Processes tokens from the Tokenizer and creates a structured representation of the expression. It must ensure that PEMDAS is followed, unary operators and parentheses are handled, and there are no invalid tokens.
  - **Key Attributes/Operations**: An attribute of tokens will be used from Tokenizer to generate a structure. A key operation includes



© University of Kansas, 2026

the parse expression method, which will parse the tokens into a structured representation.

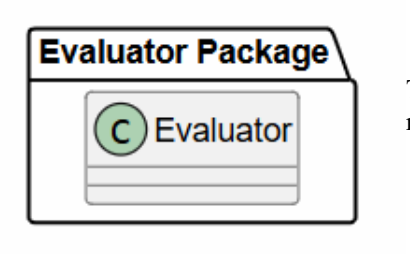
This diagram shows the Parser package containing the class responsible for processing tokens and creating a structured representation of the expression.

4.2.4 Evaluator Package

Computes the result of the arithmetic expression entered by the user. It processes the structured representation from the Parser and applies operator precedence, parentheses, and associativity to compute the result. Runtime errors are also found during this package and reported to the Error Handler.

Significant Classes:

- **Evaluator**: Traverses the structured representation and performs the necessary operations to compute the result. It ensures all operations follow the proper order, while handling runtime errors.
    - **Key Attributes/Operations**: Attributes include the structured representation from the Parser and a result attribute with the computed value. Key operations include the evaluate method, unary method for unary operations, and the apply operator method to perform arithmetic for the current operator.



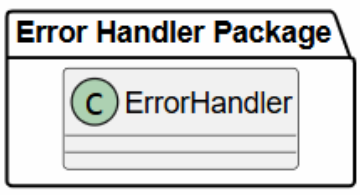
This diagram shows the Evaluator package containing the class responsible for evaluating the arithmetic expression from the user.

4.2.5 Error Handler Package

Detects and helps in the reporting process for errors that are found in other layers. Other layers will report errors that are found to this package, where it will help differentiate and handle each possible error. Then, it will be connected to the Output Handler, which focuses on outputting the error to the user through the command-line interface.

Significant Classes:

- **ErrorHandler**: Detects, classifies, and reports errors that are found during the processing of the arithmetic expression. It receives error notifications from other packages, formats clear error messages, and forwards them to the Output Handler.
    - **Key Attributes/Operations**: An error attribute will be necessary to hold the returned representation of the error. Key operations include the report error method, which formats an error for the Output Handler, and a capture error method to receive errors from other packages.



This diagram shows the Error Handler package containing its

class to detect and report errors that occur from the expression.

5. Interface Description

The system uses a simple terminal-based command-line interface for user interaction. The interaction for user input and output is illustrated in the User Interface (I/O) package described in section 4.2.

Users enter arithmetic expressions directly into the terminal, including numeric constants, operators (+, −, *, /, %, **), and parentheses. The program parses the input expression and evaluates it according to operator precedence and PEMDAS rules.

After evaluation, the result is displayed in the terminal. If the expression is invalid, such as containing incorrect syntax or division by zero, the program returns an appropriate error message to the user.

6. Quality

The architecture of the Arithmetic Expression Evaluator system supports such critical qualities as extensibility, reliability, maintainability, and usability.

Extensibility:

The system is divided into modules such as tokenizer, parser, evaluator, and program controller, which can easily be updated to add extra functionality to the program (e.g., new operation types and support for floating-point numbers).

Reliability:

The system contains proper error handling for cases such as incorrect inputs, division by zero, and other errors that might arise during the tokenization, parsing, and evaluation processes.

Maintainability:

The system follows the principles of object-oriented programming with clearly defined classes such as Token and its subclasses, which make the architecture more readable and convenient for debugging, modifying, and updating in the future.

Usability:

The system provides an easy-to-use console UI where users can type in their expression, and then, based on the program output, decide whether further calculations should be performed.

Performance:

The system performs expression evaluation efficiently, using a token-based approach and recursion to avoid extra calculations.

Portability:

The system was developed using standard C++, which means that it can run on any platform with a proper compiler installed.

 © University of Kansas, 2026